The first line of the definition is one of those Given/When/Then words (it doesn't matter which one) followed by a regular expression. Cucumber matches a step to a definition when the sentence part of the step, in this case a project, matches the regular expression—such as /^a project$/. You'll see later why Cucumber uses regular expressions instead of just strings. When Cucumber sees the step Given a project, it finds the step definition whose regular expression matches, then runs the code inside the block for the matching step definition. If Cucumber finds more than one matching step definition raises an error.

Inside a step definition you can write any arbitrary Ruby code. Instance variables declared in one step definition will be available to later step definitions in the same test. Be careful with instance variables; it's not always easy to tell what variables might exist from previous steps, or what state they might be in. Cucumber understands Capybara methods and it understands RSpec matchers (assuming RSpec is installed). Arbitrary methods defined in any step-definition file will be available to any step definition.

Your case here is unusual in that you already have this feature managed as a non-Cucumber integration test, so filling the steps is mostly a question of splitting that test into pieces. My copy looks like this:

integration/03/features/step_definitions/add_task_steps.rb

```
Given(/^a project$/) do
  @project = Project.create(name: "Bluebook")
  @project.tasks.create(title: "Hunt the aliens", size: 1, project_order: 1)
  @project.tasks.create(title: "Write a book", size: 1, project_order: 2)
end

When(/^I visit the project page$/) do
  visit project_path(@project)
end

When(/^I complete the new task form$/) do
  fill_in("Task", with: "Find UFOs")
  select("2", from: "Size")
  click_on("Add Task")
end

Then(/^I am back on the project page$/) do
  expect(current_path).to eq(project_path(@project))
end

Then(/^I see the new task is last in the list$/) do
  within("#task_3") do
    expect(page).to have_selector(".name", text: "Find UFOs")
    expect(page).to have_selector(".size", text: "2")
    expect(page).not_to have_selector("a", text: "Down")
  end
end
```